

Sistema de Tipos

Material de lectura sobre el tema: Complementar lo presentado aquí con:

- Sebesta: Capítulo 6 (desde sección 6.12)
- Scott: Capítulo 7

Observación: Para aquellos que les gusta más Sebesta, para este tema hagan el esfuerzo de complementar lo presentado aquí con el libro de Scott. Sebesta hace una presentación mucho más básica de este tema.

El sistema de tipos

Mencionamos en el documento anterior que **el sistema de tipos es un conjunto de reglas que estructuran y organizan una colección de tipos**. Estas reglas y su organización tienen un grado de variedad importante y es por eso que la parte más interesante de la discusión de tipos surge del análisis de esta parte. **El objetivo del sistema de tipos de un lenguaje es lograr que los programas sean tan seguros como sea posible.**

Informalmente el sistema de tipos consiste de:

- ✧ Un mecanismo para definir tipos y asociarlos con ciertos constructores del lenguaje (los constructores involucrados son aquellos que manejan valores o hacen referencias a objetos que tienen valores).
- ✧ Un conjunto de reglas para equivalencias de tipos, compatibilidad, inferencia, etc.

Uno de los aspectos más importantes que aparecen vinculados al sistema de tipos tiene que ver con el chequeo de tipos. Hasta el advenimiento de los lenguajes de scripting, los lenguajes tendían a tener sistemas de tipos con reglas suficientes como para poder resolver todo lo posible (en relación al control de tipos) de manera estática. Este detalle hacía que los lenguajes fueran estáticamente tipados (la ligadura del atributo tipo de las variables se establece de manera estática), tuviera sentido que fueran compilados y, por lo tanto, contarán con una semántica estática bastante fuerte. El punto a tener en cuenta en el diseño del lenguaje, era el de adelantar los chequeos de tipo lo más posible (en el proceso de reconocimiento, traducción, ejecución), bajo la premisa de que cuanto antes se pueden hacer, más eficiencia se obtiene (tanto en uso de memoria como en tiempo de ejecución).

La aparición de los lenguajes de scripting puso un poco en jaque esta premisa general y comenzó a popularizar el tipado dinámico. La premisa anterior sigue siendo válida, pero para estos lenguajes la flexibilidad en la administración de los tipos era más importante que la eficiencia. Algunos lenguajes se relajaron aún más y perdieron seguridad también. Este último punto está asociado al hecho de si los controles se hacen o no, claramente de hacerse deben hacerse en ejecución, ya que la forma de tipado no permite adelantar los controles.

Esa dicotomía está asociada al balance entre dos criterios que a veces se contraponen, la búsqueda de seguridad a veces compromete a la flexibilidad del lenguaje. Noten que realmente es “a veces” porque depende fundamentalmente de cómo se conciben los controles de seguridad. Hay una contraposición inevitable entre eficiencia y flexibilidad, el optar por garantizar en profundidad una genera un compromiso de pérdida inevitable en la otra.

Los lenguajes orientados al desarrollo de aplicaciones complejas buscan mayor seguridad y se caracterizan por tener un sistema de tipos estricto, mientras que los lenguajes orientados al desarrollo rápido, buscan mayor flexibilidad de modo que se caracterizan por tener un sistema de tipos relajado.

Para pensar: ¿Qué pasaría si quisiéramos hacer un compilador para un lenguaje con tipado dinámico?

El sistema de tipos del lenguaje se encarga de especificar reglas para:

- ✧ Tipo y tiempo de chequeo: el chequeo de tipo consiste en verificar que el tipo de una entidad corresponda al esperado por el contexto.
- ✧ Reglas de equivalencia y conversión: en caso de que los tipos de las entidades no coincidan con lo esperado, puede tratarse de un error o se aplican reglas de coerción y/o reglas de equivalencia.
- ✧ Reglas de inferencia de tipo.
- ✧ Nivel de polimorfismo del lenguaje.

Dureza del tipado (strong typing)

Nota: Si bien la traducción más directa es hablar de tipado fuerte, nosotros en la materia preferimos utilizar *dureza* para evitar confusiones con respecto a la propiedad de fuertemente tipado o *strongly typed language*.

La dureza del tipado no es una propiedad de los lenguajes, sino que **es un grado que surge como consecuencia de las restricciones que impone el sistema de tipos** para cada uno de los ítems mencionados en el punto anterior. En particular, el análisis al que apuntamos desde la materia es determinar el grado de dureza del lenguaje respecto a una característica, el cual está determinado por cómo se incorpora y/o se manipula esa característica en el lenguaje.

Por ejemplo, podemos analizar el grado de dureza del sistema tipos con respecto a las expresiones mixtas:

Lenguaje con Tipado menos duro	Lenguaje con Tipado más duro
A = 2	A = 2
B = "2"	B = "2"
Concatenar(A,B) <i>{retorna "22"}</i>	Concatenar(A,B) <i>{error de tipos}</i>
Sumar(A,B) <i>{retorna 4}</i>	Sumar(A,B) <i>{error de tipos}</i>
	Concatenar(srt(A),B) <i>{retorna "22"}</i>
	Sumar(A,int(B)) <i>{retorna 4}</i>

Un lenguaje más duro exige que el programador establezca qué operación es la que quiere aplicar, mientras que en uno más débil las transformaciones de tipo ocurren de manera implícita para poder aplicar el operador sobre tipos no esperados sobre la misma.

Lenguaje Fuertemente tipado (strongly typed language)

Esta propiedad garantiza lo que se conoce como *type safety*. Para que un lenguaje satisfaga esta propiedad debe **detectar todos los errores de tipos**. Si el lenguaje maneja muy bien los tipos, pero hay un detalle que no es capaz de detectar entonces el lenguaje pierde la propiedad, el lenguaje es o no es fuertemente tipado, no hay grises aquí.

Un detalle importante a tener en cuenta es que **no importa en qué momento detecte el error de tipos**, lo importante es que lo detecte y permita al programador poder hacer algo con ese error para que la aplicación que está desarrollando no colapse por un error de tipos.

El tiempo de ligadura a los tipos genera variantes en el momento en el que los controles se hacen y tiene un impacto sobre la eficiencia, pero es importante tener en cuenta que para contar con la propiedad no se tiene en cuenta la eficiencia sino que los controles se hagan. De todos modos, hay características de los lenguajes que no son controlables estáticamente, aunque el lenguaje tenga tipado estático. ¿Un ejemplo? El acceso a las componentes de un registro variante o a una estructura dinámica.

Reglas de conversión

Un sistema de tipos flexible brinda reglas de conversión o coerción que permiten aceptar datos de un tipo Q en un contexto en el cual se espera un dato de tipo T. **La conversión puede ser con pérdida (limitante) o sin pérdida (expansora).**

Las conversiones limitantes se consideran *inseguras*, porque se pierde información que no puede recuperarse. Los lenguajes tienden a no utilizarlas de manera implícita a menos que sea algo inevitable (por ejemplo, en una asignación cuando el tipo de la variable destino es más limitado que el tipo de la expresión asignada).

Las conversiones expansoras se consideran *seguras*, precisamente porque no generan pérdida de información, solamente generan una modificación en la forma en la que un determinado valor está almacenado (en el caso de los lenguajes convencionales).

Sea $f: T1 \rightarrow R1$

La operación f puede ser invocada con un argumento de tipo $T2$, si el lenguaje brinda una regla que permita convertir los valores de tipo $T2$ en valores de tipo $T1$.

La operación f puede ser invocada en un contexto en el que se espera un valor de tipo $R2$, si el lenguaje brinda reglas para convertir valores de tipo $R1$ en valores de tipo $R2$.

Reglas de equivalencia

En este caso no hay conversión de tipos. Dos tipos resultan equivalentes si cada uno de ellos puede aparecer en un contexto en el que opera una aparición del otro sin requerir de ningún cambio.

Es importante tener en cuenta que en un lenguaje fuertemente tipado las reglas de equivalencia deben quedar establecidas en el diseño del mismo.

Los lenguajes suelen apelar a algunas de las siguientes formas de establecer equivalencias de tipos:

- ✧ **Equivalencia por nombre:** dos variables son de tipos equivalentes si han sido declaradas en una misma instrucción o con exactamente el mismo nombre de tipo.
- ✧ **Equivalencia por declaración:** dos variables son equivalentes si conducen a la misma expresión de tipo original luego de una serie de re-declaraciones.
- ✧ **Equivalencia por estructura:** dos tipos de datos son equivalentes si están definidos por dos expresiones de tipo idénticas.

Inferencia de tipos

La inferencia de tipos permite que el tipo de una entidad declarada se “infiera” en lugar de ser declarado. Los algoritmos que utilizan los lenguajes con esta particularidad son complejos pero están basados en la siguiente idea:

La inferencia puede realizarse de acuerdo al tipo de:

- ✧ Un operador predefinido $\text{fun } f1(n,m) = (n \bmod m = 0)$
- ✧ Un operando $\text{fun } f2(n) = (n * 2)$
- ✧ Un argumento $\text{fun } f3(n:\text{int}) = n * n$
- ✧ El tipo del resultado $\text{fun } f4(n):\text{int} = (n * n)$

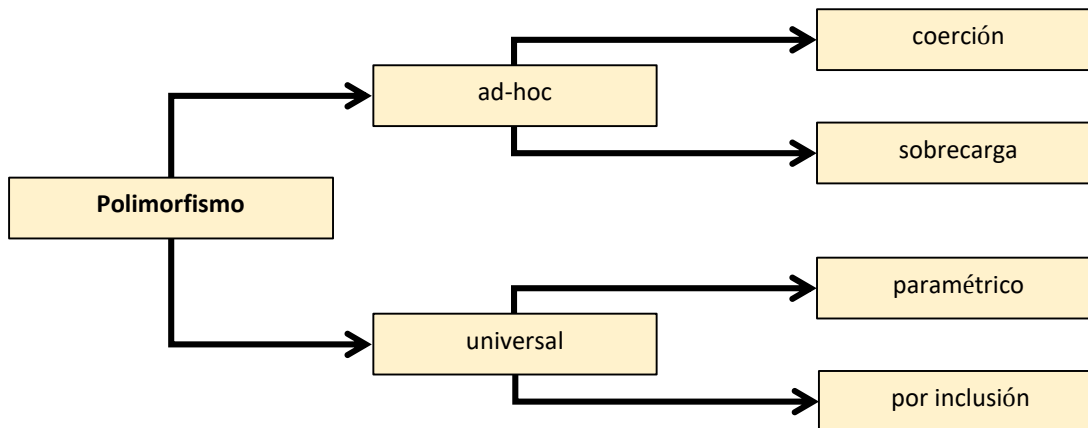
Niveles de polimorfismo

Los lenguajes pueden aportar desde su definición diferentes grados de polimorfismo. Pueden tener todos los niveles, algunos o solo uno. El gráfico que está más abajo muestra la taxonomía vinculada.

Las opciones del llamado **polimorfismo ad-hoc** están dentro de la taxonomía, pero se trata de un nivel aparente de polimorfismo. Es aparente porque es una ilusión que se le vende al programador. Internamente esas entidades no son polimórficas.

Tengan en cuenta que por definición algo es polimórfico si la misma entidad puede tomar diferentes “formas”, en el caso de los lenguajes diferentes tipos. Para que una entidad sea polimórfica debe poder estar ligada a más de un tipo. Así, las variables polimórficas pueden tomar valores de diferentes tipos, las operaciones polimórficas son funciones que aceptan operandos de varios tipos y los tipos polimórficos tienen operaciones polimórficas.

Para entender por qué las operaciones sobrecargadas no son estrictamente polimórficas, la clave está en lo que definimos recién. No hay una operación que recibe operandos de distintos tipos, sino varias operaciones monomórficas que coinciden en el nombre.



Detalles de la sobrecarga

La sobrecarga necesita criterios definidos en el sistema de tipos para resolverse. Puede aplicarse de dos maneras:

- ✧ Independientes del contexto (los datos de entrada de la función son de tipos diferentes).
- ✧ Dependientes del contexto (los datos de entrada son del mismo tipo, no así los de salida).

La incorporación de sobrecarga al lenguaje puede traer inconvenientes si se combina con otras características como el tipo unión, la utilización de parámetros por omisión y las expresiones mixtas. En general los lenguajes están obligados a recortar el poder expresivo de alguna de las características mencionadas para poder resolver satisfactoriamente cualquier expresión (sobrecarga, coerciones implícitas, expresiones mixtas, parámetros por omisión).

Detalles del polimorfismo por inclusión

En un lenguaje orientado a objetos con un sistema fuertemente tipado el polimorfismo de las variables se ve limitado. El costo es la integridad conceptual del lenguaje que se ve comprometida al imponer fuertes restricciones sobre la herencia.

Para evitar las violaciones de tipos se debe poder sustituir a un objeto de una clase derivada por un objeto de la clase base en cualquier contexto. Si se puede garantizar la sustitución de objetos, un tipo derivado puede verse como un subtipo del tipo base. Imponiendo ciertas restricciones en el uso de la herencia se puede garantizar la sustitución. Estas restricciones son las siguientes:

- ✧ **Extensión del tipo:** La clase derivada solo puede extender la funcionalidad de la clase base. Pero no puede modificar o esconder las operaciones provistas por la clase base. Este mecanismo fue adoptado por Ada 95.
- ✧ **Sobre-escritura de las operaciones:** Permite que la clase derivada redefina una operación heredada. En este caso, para garantizar la sustitución de objetos debemos:
 1. Los parámetros de entrada deben ser super-tipos de los tipos de los parámetros originales de la operación.
 2. Los parámetros de salida deben ser sub-tipos de los tipos de los parámetros originales de la operación.

Lo que hacen básicamente es garantizar que la relación de herencia sea una relación de tipo *is-a*. Lamentablemente esto se vuelve bastante difícil ya que la mayoría de los lenguajes brindan un constructor adecuado para herencia de software, con lo que eso implica.

Para los lenguajes con tipado estático, la propiedad de fuertemente tipado está asociada a un grado de dureza alto en las características vinculada a polimorfismo.